

05_shap_explainability

July 20, 2026

1 05 — Phase 3: SHAP explainability + context/skill-driver validation

Purpose: rebuild the Phase 1/2 pipeline exactly as in notebooks 03/04, then run the new `src/models/explainability.py` layer against real data. Three questions this notebook needs to answer honestly, not assume:

1. Does per-prediction SHAP additivity actually hold on real held-out rows, not just the synthetic tests in `tests/test_explainability.py`?
2. How much of the model's total `|SHAP|` mass sits in `percentile/job_zone` (context) versus actual skills — this determines whether a naive “top feature importances” list would mislead a lay user into thinking distributional position is a skill.
3. Do the local explanations for previously-unmatched occupations (same spot-checks as notebooks 03/04) look plausible, and does a matched occupation with a real BLS wage look plausible too?

```
[1]: import sys
from pathlib import Path
project_root = Path.cwd().parent
sys.path.insert(0, str(project_root))

import numpy as np
import pandas as pd

from src.data.onet_loader import (
    load_occupation_data, load_essential_skills, load_transferable_skills,
    load_job_zones, build_full_skill_importance_matrix,
)
from src.data.oews_loader import load_national_wages
from src.features.occupation_features import (
    build_occupation_feature_table, build_training_rows, clean_training_rows,
)
from src.models.wage_model import occupation_train_test_split,
    ↪train_baseline_model
from src.models.explainability import (
    build_explainer, explain_prediction, global_feature_importance,
    ↪CONTEXT_FEATURES,
)
```

```
/Users/ayaangubitra/Desktop/University/London School of  
Economics/Others/Projects/career-intelligence-system/.venv/lib/python3.13/site-  
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update  
jupyter and ipywidgets. See  
https://ipywidgets.readthedocs.io/en/stable/user\_install.html  
from .autonotebook import tqdm as notebook_tqdm
```

1.1 1. Rebuild the Phase 1 pipeline

Identical to notebooks 03/04 – same loaders, same feature table, same melted training rows, same `test_size=0.2`, `random_state=42` split. This is deliberate: explainability sits on top of the existing pipeline, so the model being explained here should be the exact same model (same rows in train/test) already validated in 03 and 04.

```
[2]: occ = load_occupation_data()
skills = load_essential_skills()
transferable = load_transferable_skills()
job_zones = load_job_zones()
wages = load_national_wages()

skill_matrix = build_full_skill_importance_matrix(skills, transferable)
skill_cols = list(skill_matrix.columns)
feature_table = build_occupation_feature_table(occ, skill_matrix, job_zones,
↪wages)
training_rows = build_training_rows(feature_table, skill_cols=skill_cols)
cleaned_training_rows = clean_training_rows(training_rows,
↪skill_cols=skill_cols)

feature_cols = skill_cols + ["job_zone", "percentile"]
train_df, test_df = occupation_train_test_split(cleaned_training_rows,
↪test_size=0.2, random_state=42)
model = train_baseline_model(train_df, feature_cols)

print(f"Train occupations: {train_df['onetsoc_code'].unique()}
↪({len(train_df)} rows)")
print(f"Test occupations: {test_df['onetsoc_code'].unique()} ({len(test_df)}
↪rows)")
```

[oews_loader] 7 rows have suppressed wage data – kept as NaN, not dropped.
Dropped 470 rows (94 occupations) missing all skill values, out of 4780 total rows.

No job_zone imputation needed.

Train occupations: 689 (3445 rows)

Test occupations: 173 (865 rows)

1.2 2. Build one explainer, verify additivity holds on real held-out rows

`build_explainer()` is deliberately separate from `explain_prediction()` so it's built ONCE and reused – rebuilding a `TreeExplainer` per row would be wasteful across 865 test rows. Additivity

is checked per-row inside `explain_prediction`, not assumed; here we check it across the WHOLE test set at once, since a single passing row wouldn't rule out edge-case failures elsewhere in the distribution.

```
[3]: explainer = build_explainer(model)

additivity_errors = []
for i in range(len(test_df)):
    row = test_df.iloc[i]
    feature_values = {col: row[col] for col in feature_cols}
    result = explain_prediction(model, explainer, feature_values, feature_cols,
    ↪skill_cols, top_n=None)
    additivity_errors.append(result.additivity_max_abs_error)

additivity_errors = np.array(additivity_errors)
print(f"Additivity check across all {len(test_df)} test rows:")
print(f"  max error: {additivity_errors.max():.2e}")
print(f"  mean error: {additivity_errors.mean():.2e}")
print(f"  all within tolerance (<1e-3): {(additivity_errors < 1e-3).all()}")
```

Additivity check across all 865 test rows:

```
max error: 1.24e-05
mean error: 4.95e-06
all within tolerance (<1e-3): True
```

1.3 3. Global feature importance – and the context-vs-skill split

This is the aggregate honesty check described in the module docstring: if percentile/job_zone dominate the top of a naive combined ranking, that's a real finding to report, not something to reclassify away.

```
[4]: global_imp = global_feature_importance(explainer, test_df[feature_cols],
    ↪feature_cols, skill_cols, top_n=15)
global_imp
```

```
[4]:
```

	rank	feature	mean_abs_shap_log	category
0	1	percentile	0.272350	context
1	2	job_zone	0.081832	context
2	3	Critical Thinking	0.068142	skill
3	4	Complex Problem Solving	0.057607	skill
4	5	Judgment and Decision Making	0.043508	skill
5	6	Science	0.031150	skill
6	7	Systems Analysis	0.028570	skill
7	8	Reading Comprehension	0.028152	skill
8	9	Service Orientation	0.027103	skill
9	10	Systems Evaluation	0.026565	skill
10	11	Operations Analysis	0.019923	skill
11	12	Operations Monitoring	0.018045	skill

12	13	Persuasion	0.016788	skill
13	14	Management of Financial Resources	0.015672	skill
14	15	Coordination	0.015228	skill

```
[5]: full_imp = global_feature_importance(explainer, test_df[feature_cols],
    ↪ feature_cols, skill_cols, top_n=None)
total_mass = full_imp["mean_abs_shap_log"].sum()
context_mass = full_imp[full_imp["category"] == "context"]["mean_abs_shap_log"].
    ↪ sum()
context_share = context_mass / total_mass

print(f"Share of total |SHAP| mass in context features (percentile + job_zone):")
    ↪ {context_share:.1%}")
print(f"\nTop 5 SKILL-ONLY drivers (context excluded):")
print(full_imp[full_imp['category'] == 'skill'].head(5).to_string(index=False))
```

Share of total |SHAP| mass in context features (percentile + job_zone): 38.1%

Top 5 SKILL-ONLY drivers (context excluded):

rank	feature	mean_abs_shap_log	category
3	Critical Thinking	0.068142	skill
4	Complex Problem Solving	0.057607	skill
5	Judgment and Decision Making	0.043508	skill
6	Science	0.031150	skill
7	Systems Analysis	0.028570	skill

1.4 4. Local explanations – same spot-check occupations as notebooks 03/04

Reusing the previously-unmatched occupations (Buyers/Purchasing Agents, Appraisers) keeps one consistent thread running through 03 -> 04 -> 05: point estimate, then uncertainty interval, then explanation, for the same real occupations each time.

```
[6]: no_wage_match = feature_table[
    feature_table["a_median"].isna() & feature_table[skill_cols].notna().
    ↪ all(axis=1)
]
sample = no_wage_match.head(3)

for onetsoc_code, row in sample.iterrows():
    feature_values = {col: row[col] for col in feature_cols if col !=
    ↪ "percentile"}
    feature_values["percentile"] = 50 # median, for a representative
    ↪ single-row explanation
    result = explain_prediction(model, explainer, feature_values, feature_cols,
    ↪ skill_cols, top_n=5)

    print(f"\n{row['title']} ({onetsoc_code}) -- median percentile:")
```

```

    print(f" Predicted wage: ${result.predicted_wage:,.0f} (base_wage:␣
↪${result.base_wage:,.0f})")
    print(f" Additivity OK: {result.additivity_ok} (max_abs_error={result.
↪additivity_max_abs_error:.2e})")
    print(f" Context: percentile {result.context['percentile'].pct_effect:+.
↪1%}, "
          f"job_zone {result.context['job_zone'].pct_effect:+.1%}")
    print(f" Top skill drivers:")
    for e in result.skill_drivers:
        print(f"      {e.name}: {e.pct_effect:+.1%}")

```

Buyers and Purchasing Agents, Farm Products (13-1021.00) -- median percentile:

Predicted wage: \$76,438 (base_wage: \$69,764)

Additivity OK: True (max_abs_error=4.77e-06)

Context: percentile -1.4%, job_zone +8.7%

Top skill drivers:

Critical Thinking: +4.3%

Management of Financial Resources: +3.9%

Persuasion: +3.2%

Systems Evaluation: -2.9%

Science: -2.5%

Wholesale and Retail Buyers, Except Farm Products (13-1022.00) -- median percentile:

Predicted wage: \$72,266 (base_wage: \$69,764)

Additivity OK: True (max_abs_error=4.77e-06)

Context: percentile -1.3%, job_zone -2.1%

Top skill drivers:

Persuasion: +10.6%

Critical Thinking: -6.2%

Management of Financial Resources: +5.9%

Reading Comprehension: -3.4%

Systems Evaluation: +3.4%

Purchasing Agents, Except Wholesale, Retail, and Farm Products (13-1023.00) -- median percentile:

Predicted wage: \$96,025 (base_wage: \$69,764)

Additivity OK: True (max_abs_error=1.05e-05)

Context: percentile -0.2%, job_zone +8.4%

Top skill drivers:

Complex Problem Solving: +9.3%

Persuasion: +8.8%

Management of Material Resources: -7.6%

Management of Financial Resources: +5.3%

Judgment and Decision Making: +5.1%

1.5 5. Sanity check – a matched occupation with a REAL BLS wage

The spot-checks above have no ground truth to compare against by definition (that's why they needed a model in the first place). As a contrasting lens, explain a couple of occupations that DO have a real BLS wage, so the top skill drivers can be eyeballed against domain intuition for a role whose actual wage is independently known.

```
[7]: wage_matched = test_df.merge(
    feature_table[["title"]], left_on="onetsoc_code", right_index=True
).drop_duplicates("onetsoc_code")

sample_matched = wage_matched.sample(2, random_state=42)
for _, row in sample_matched.iterrows():
    feature_values = {col: row[col] for col in feature_cols if col != "percentile"}
    feature_values["percentile"] = 50
    result = explain_prediction(model, explainer, feature_values, feature_cols, skill_cols, top_n=5)

    actual_median = feature_table.loc[row["onetsoc_code"], "a_median"]
    print(f"\n{row['title']} ({row['onetsoc_code']}):")
    print(f"    Actual BLS median wage: ${actual_median:.0f}")
    print(f"    Model predicted (median percentile): ${result.predicted_wage:.0f}")
    print(f"    Top skill drivers:")
    for e in result.skill_drivers:
        print(f"        {e.name}: {e.pct_effect:+.1%}")
```

```
Light Truck Drivers (53-3033.00):
Actual BLS median wage: $44,860
Model predicted (median percentile): $44,882
Top skill drivers:
Critical Thinking: -6.8%
Complex Problem Solving: -6.1%
Judgment and Decision Making: -6.0%
Service Orientation: +3.8%
Systems Evaluation: -3.1%
```

```
Chemical Technicians (19-4031.00):
Actual BLS median wage: $60,390
Model predicted (median percentile): $76,142
Top skill drivers:
Critical Thinking: +8.3%
Science: +7.2%
Complex Problem Solving: -4.7%
Systems Evaluation: +4.3%
Judgment and Decision Making: -3.5%
```

1.6 6. Verdict

- **Additivity:** holds cleanly across all 865 held-out test rows — max error 1.24e-05, mean error 4.95e-06, both far inside the 1e-3 tolerance. No sign of the monotonic-constraint additivity risk flagged going into this phase.
- **Context vs. skill split:** percentile and job_zone together account for 38.1% of total |SHAP| mass, with percentile alone outranking every individual skill. This confirms the context/skill-driver split wasn't a nice-to-have: a naive “top 5 feature importances” list would show “50th percentile” as the single largest driver, which is meaningless to a lay user. Once separated out, the skill-only ranking is coherent — Critical Thinking, Complex Problem Solving, and Judgment and Decision Making top the list.
- **Local explanations:** the three previously-unmatched spot-check occupations produce plausible, differentiated skill drivers (e.g. Persuasion and Management of Financial Resources pulling purchasing-agent estimates up). The real-wage sanity check is more mixed: Light Truck Drivers predicted almost exactly on target (\$44,882 vs. actual \$44,860), but Chemical Technicians overpredicted by ~26% (\$76,142 vs. actual \$60,390), driven up by Science and Critical Thinking. Worth stating plainly rather than glossing over — the explanation at least makes the overprediction interpretable, which is the actual value of this phase even when the point estimate misses.

Phase 3 verdict: proceed to Phase 4 (API + tests). The Chemical Technicians gap is worth a one-line mention in the README's Limitations section, in the same honest spirit as the Job Zone 4 boundary case and the percentile-level coverage decline from Phase 2.